The Modern Default

Adam, AdamW & the Whole Family

Lesson 5 of the Optimizer Course

The optimizer that trains almost everything —
built from the two ideas you already know

This is the big one. Adam is not a new idea — it's the marriage of the two ideas you just learned: momentum's direction-memory and RMSProp's per-weight step sizes, with one neat fix on top. We'll build it piece by piece, do the arithmetic by hand, understand the "bias correction" that confuses everyone, learn why AdamW *(not Adam) is the true default, and meet the whole family of variants in plain language.*

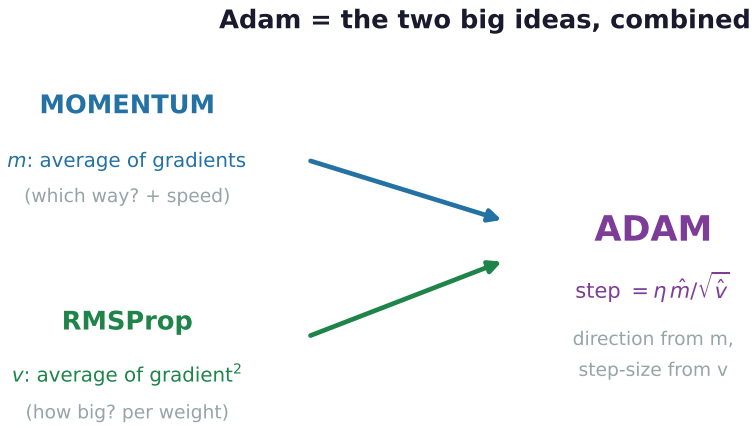
Marevlo Research

1 Adam is just two old friends, holding hands

You already have both halves of Adam:

- From **Lesson 3 (momentum):** keep a running average of the gradients — this tells you a reliable *direction* and builds speed. Call it m (the "first moment" — the mean).
- From **Lesson 4 (RMSProp):** keep a running average of the gradients *squared* — this tells you each weight's *scale*, so you can size its step. Call it v (the "second moment").

Adam uses *both at once*: move in the smoothed direction m, with a per-weight step size set by v. That's the whole thing.



$$m \leftarrow \beta_1 m + (1-\beta_1) g \qquad v \leftarrow \beta_2 v + (1-\beta_2) g^2$$

$$\hat{m} = \frac{m}{1-\beta_1^t}, \qquad \hat{v} = \frac{v}{1-\beta_2^t}, \qquad w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

Say it in English: (1) update the direction-average m; (2) update the scale-average v; (3) apply a "bias correction" (next section); (4) step in direction m, with each weight's step shrunk by its own scale v.

The defaults — which you'll see everywhere — are $\beta_1 = 0.9$ (direction memory ≈ 10 steps), $\beta_2 = 0.999$ (scale memory ≈ 1000 steps), $\eta = 10^{-3}$, $\varepsilon = 10^{-8}$.

2 The bias correction, finally explained

Everyone trips on the $\frac{1}{1-\beta^t}$ terms. Here's the simple truth. Both m and v start at *zero*. So on the very first step they're badly pulled *toward* zero — they under-report. Watch:

Let's actually do the numbers

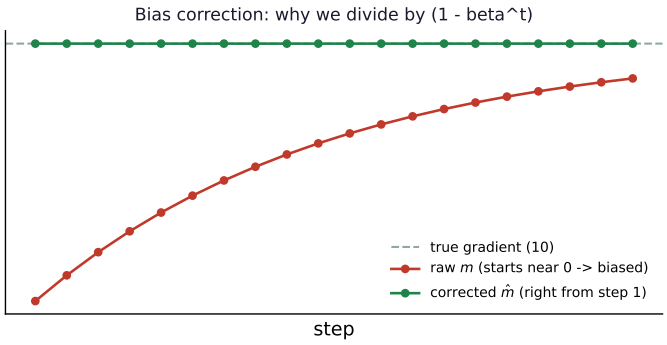
Step 1, first gradient $g = 10$ (taking one weight). With $\beta_1 = 0.9$:

$$m = 0.9 \cdot 0 + 0.1 \cdot 10 = 1.0.$$

But the "average gradient so far" should just be 10 — we only have one! Instead $m = 1$, a **tenfold under-estimate**, purely because it started at zero. The fix: divide by $(1 - \beta_1^t)$. At $t = 1$ that's $(1 - 0.9) = 0.1$, so

$$\hat{m} = \frac{1.0}{0.1} = 10 = \text{the true gradient.} \quad \text{Fixed exactly.}$$

As t grows, $\beta_1^t \to 0$, so the correction factor $\to 1$ and quietly switches itself off once enough history has built up. It only matters early.



Without this fix, Adam would take tiny, timid steps for the first dozen-or-so iterations (because m and v are artificially small). The correction makes Adam confident from step one.

3 Watch Adam run (by hand)

Same stretched bowl, $\nabla f = (10w_1, w_2)$, start $(1, 1)$, defaults above.

Let's actually do the numbers

Step 1: $g = (10, 1)$. $m = (1.0, 0.1)$, $v = (0.1, 0.001)$. Correct: $\hat{m} = m/0.1 = (10, 1)$, $\hat{v} = v/0.001 = (100, 1)$. Step $= 0.1 \cdot \frac{(10,1)}{(\sqrt{100},\sqrt{1})} = 0.1 \cdot (1, 1) = (0.1, 0.1)$. **Both weights move** 0.1 — direction from m, equalised step from v. $w = (0.9, 0.9)$.

Step 2: $g = (9, 0.9)$. $m = (1.8, 0.18)$, $v = (0.181, 0.00181)$. With $t = 2$ corrections $(1-\beta_1^2 = 0.19,\ 1-\beta_2^2 = 0.002)$: $\hat{m} = (9.47, 0.95)$, $\hat{v} = (90.5, 0.905)$. Step $= 0.1 \cdot \frac{(9.47,0.95)}{(9.51,0.951)} = (0.0996, 0.0995)$. $w = (0.800, 0.800)$.

Adam marches both coordinates down by ≈ 0.1 every step — smooth, equalised, no zig-zag, no hand-tuning per direction. *This* is why it became the default: it mostly just works.

4 AdamW: the fix that makes it the *real* default

There's a subtle bug in "plain" Adam the moment you add **weight decay** (the standard regularisation that gently pulls weights toward zero to prevent over-fitting). The old way ("L2

regularisation") adds a λw term *into the gradient*. But then that decay term gets run through Adam's per-weight scaling $\sqrt{\hat{v}}$ — so weights with big gradients get their decay shrunk away, and weights with small gradients get hammered. The decay becomes uneven and basically random.

Let's actually do the numbers

Two weights, both $= 1$, decay $\lambda = 0.01$, $\eta = 0.1$.

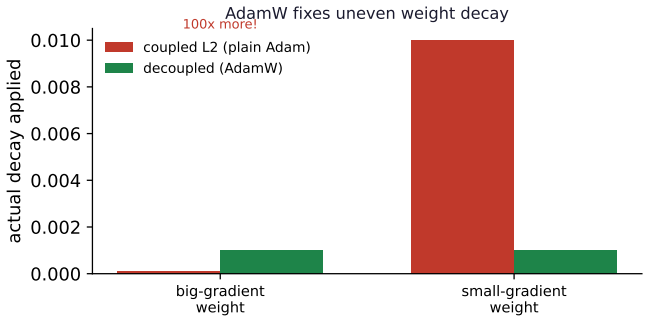
- A weight with **big** gradients ($\sqrt{\hat{v}} = 10$): coupled decay $= \frac{\eta\lambda w}{\sqrt{\hat{v}}} = \frac{0.1 \cdot 0.01}{10} = 0.0001$.
- A weight with **small** gradients ($\sqrt{\hat{v}} = 0.1$): coupled decay $= \frac{0.1 \cdot 0.01}{0.1} = 0.01$.

Same intended decay, but one weight decays **100× more** than the other — nonsense.

AdamW's fix: apply the decay *directly to the weight*, outside the adaptive scaling:

$$w \leftarrow w - \eta\left(\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon} + \lambda w\right).$$

Now both weights decay by $\eta\lambda w = 0.1 \cdot 0.01 \cdot 1 = 0.001$ — **equal, as intended.** "W" stands for this decoupled **W**eight decay.



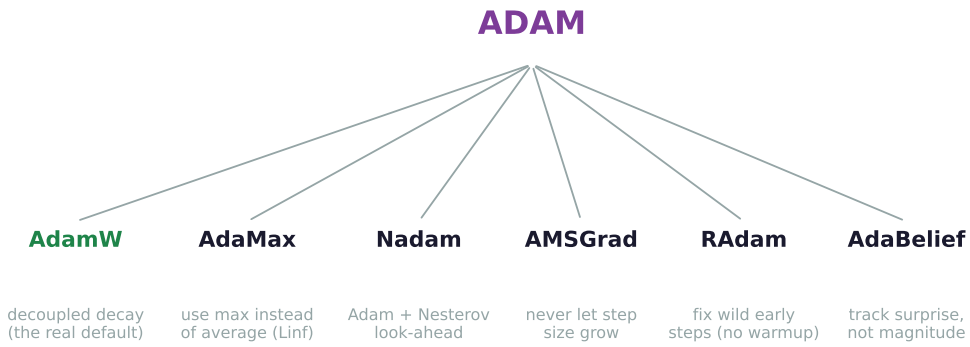
When you're actually training

Use AdamW, not Adam, for anything modern — every serious transformer / LLM is trained with it. In most libraries it's a one-word change. Plain Adam's coupled decay genuinely hurts large models. If you remember one practical thing from this lesson: *the default optimizer is AdamW.*

5 The family tree: seven variants in plain words

Once you have Adam, a swarm of variants follows, each tweaking one piece. You don't need to memorise them — just know what knob each turns.

- **AdaMax.** Replaces the squared-gradient average v with a running *maximum* of the gradient size (the "infinity norm"). Because a max is less jumpy than an average of squares, it can be steadier when gradients occasionally spike — handy for some embedding layers.
- **Nadam.** Adam with **Nesterov**'s look-ahead (Lesson 3) bolted onto the momentum term. Same anticipate-the-overshoot benefit, slightly snappier convergence.
- **AMSGrad.** Forces the scale term to use the *largest* $\hat{v}$ seen so far, so a weight's effective step size can never *increase* over time. This patched a hole in Adam's original convergence proof. Mathematically tidy; in practice it rarely changes much.



- **RAdam (Rectified Adam).** Early in training, v is averaged over just a few gradients, so the per-weight scale is wildly unreliable — which is exactly why people add "warmup" (Lesson 9). RAdam computes how trustworthy v is and automatically *turns off* the adaptive part until enough data has accumulated, often removing the need for warmup entirely.
- **AdaBelief.** Instead of scaling by how *big* gradients are (g^2), it scales by how *surprising* they are — the variance of g around its own moving average m, i.e. $(g-m)^2$. Steady, predictable gradients get big confident steps; erratic ones get cautious steps. Claims Adam-like speed with SGD-like generalisation.
- **AdamW** (already covered). The decoupled-decay version — and the one that actually matters in practice.

Watch out (common confusion)

A trap worth naming: Adam and friends often reach low *training* loss fastest, but plain **SGD with momentum** sometimes generalises better (lower *test* error), especially in vision. "Fastest to train" is not "best final model." Many strong image results still use SGD+momentum precisely for that reason (we'll see why in Lesson 8 on flat minima). Adam's real home turf is transformers, NLP, and anything with sparse or wildly-scaled gradients.

6 What you can do now, and what's next

You can now build Adam from momentum + RMSProp, explain and compute the bias correction, run Adam by hand, explain why AdamW's decoupled decay is the correct default, and say in one line what each of the seven variants changes.

Next — Lesson 6: Training Giants. Notice Adam stores *two* extra numbers (m and v) for *every single weight*. For a billion-parameter model that's billions of extra numbers just for the optimizer — often more memory than the model itself. When you're training something enormous, that becomes the bottleneck. Lesson 6 is about the optimizers built to survive it: **AdaFactor, LAMB, LARS, and NovoGrad**.